CH3 C 转 MIPS 体系化练习

覆盖：PPT CH3 全部 C 转 MIPS 相关知识点 + 真题风格训练 + 答案解析

使用说明: 本练习只使用复习 PPT 中出现的核心 MIPS 指令：add, sub, lw, sw, addi, slti, andi, ori, sll, srl, and, or, nor, beq, bne, slt, j, jal, jr。函数调用部分会用到 $a0-$a3, $v0-$v1, $ra, $sp；不使用 move/li/blt/bgt 等伪指令。

建议顺序：先做第 1-20 题打基础，再做数组和分支，第 41 题以后按真题应用题标准写注释。答案在后半部分，练习时先遮住答案。

一、做题前必须会的固定模板

总原则: C 转 MIPS 的核心原则：内存中的数不能直接参加 add/sub/and/or 等运算；必须先 lw 到寄存器，算完需要写回内存时再 sw。

1. 指令速查表

功能	指令	R/I/J	典型格式	含义
算术	add	R	add $s0,$s1,$s2	$s0 = $s1 + $s2
算术	sub	R	sub $s0,$s1,$s2	$s0 = $s1 - $s2
数据传输	lw	I	lw $s0,32($s4)	$s0 = Memory[$s4+32]
数据传输	sw	I	sw $s0,48($s4)	Memory[$s4+48] = $s0
立即数	addi	I	addi $s0,$s1,-1	$s0 = $s1 - 1
立即数	slti	I	slti $s0,$s1,10	$s0 = ($s1 < 10)
立即数	andi	I	andi $s0,$s1,10	$s0 = $s1 & 10
立即数	ori	I	ori $s0,$s1,10	$s0 = $s1 | 10
逻辑	sll/srl	R	sll $s0,$s1,2	左/右移位
逻辑	and/or/nor	R	nor $s0,$s1,$s2	$s0 = ~($s1 | $s2)
分支	beq/bne	I	beq $s0,$s1,L1	相等/不等则跳转
比较	slt	R	slt $t0,$s1,$s2	$t0 = ($s1 < $s2)
跳转	j	J	j Loop	无条件跳转
调用/返回	jal/jr	J/R	jal f ; jr $ra	调用函数/返回

2. 三个最重要的翻译模板

```
# 模板 A：表达式在寄存器里
# C: x = y + z
add $s0, $s1, $s2        # $s0 = $s1 + $s2

# 模板 B：固定下标 word 数组访问
# C: x = A[8]       A基地址在 $s4
lw  $s0, 32($s4)        # 32 = 8 * 4 bytes

# 模板 C：变量下标 word 数组访问
# C: x = A[i]       i在 $s1，A基地址在 $s4
sll $t0, $s1, 2         # $t0 = i * 4
add $t0, $s4, $t0       # $t0 = &A[i]
lw  $s0, 0($t0)         # $s0 = A[i]
```

易错点: MIPS 地址按 byte 编址；lw/sw 一次访问 1 word = 4 bytes。因此 word/int 数组 A[i] 的字节偏移量是 i*4。

3. 条件与循环模板

```
# if ($s1 == $s2) goto L1
beq $s1, $s2, L1

# if ($s1 != $s2) goto L1
bne $s1, $s2, L1
```

```
# if ($s1 < $s2) goto L1
slt $t0, $s1, $s2       # $t0 = ($s1 < $s2)
bne $t0, $zero, L1      # true -> jump

# while ($s0 < $s1) { body; $s0++; }
Loop:
    slt  $t0, $s0, $s1  # i < n ?
    beq  $t0, $zero, Exit
    # body
    addi $s0, $s0, 1
    j    Loop
Exit:
```

二、体系化练习题（先做题，后看答案）

A 组：单句表达式和立即数（目标：熟悉 add/sub/addi/slti/andi/ori/sll/srl/and/or/nor ）

本组约定: 约定：$s0, $s1, $s2, $s3 分别保存长期变量；$t0-$t3 可作临时寄存器。不要使用伪指令。

1. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 + $s2

_____

_____

2. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 - $s2

_____

_____

3. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 + 15

_____

_____

4. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 - 7

_____

_____

5. 把下面的寄存器表达式翻译成 MIPS：$s0 = ($s1 < 100)

_____

_____

6. 把下面的寄存器表达式翻译成 MIPS：$s0 = ($s1 < -3)

_____

_____

7. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 & 0x00ff

_____

_____

8. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 | 0x000f

_____

_____

9. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 << 2

_____

_____

10. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 >> 3

_____

_____

11. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 & $s2

_____

_____

12. 把下面的寄存器表达式翻译成 MIPS：$s0 = $s1 | $s2

_____

_____

13. 把下面的寄存器表达式翻译成 MIPS：$s0 = ~($s1 | $s2)

14. 把下面的寄存器表达式翻译成 MIPS：$s0 = ~$s1，要求只用 nor 和 $zero

15. 把下面的寄存器表达式翻译成 MIPS：$s0 = ($s1 + $s2) - ($s3 + 5)，要求用临时寄存器

B 组：固定下标数组访问（目标：掌握 byte addressing 与偏移量 i*4 ）

本组约定: 约定：A 的基地址在 $s4，B 的基地址在 $s5。数组元素均为 word/int。$s0-$s3 是变量，$t0-$t3 是临时寄存器。

16. 把 A[0] 读入 $s0

17. 把 A[3] 读入 $s0

18. 把 $s0 写入 A[5]

19. $s0 = A[8] + $s1

20. A[12] = $s0 - A[4]

21. $s0 = A[1] + A[2]

22. A[6] = A[6] + 1

23. B[4] = A[4]

24. A[10] = B[1] + B[2]

25. 指出并改正：add $s0,$s1,32($s4)

C 组：变量下标数组访问（目标：用 sll 计算 i*4，再用 add 求地址）

本组约定: 约定：i 在 $s1，j 在 $s2，A 基地址在 $s4，B 基地址在 $s5，word 数组。

26. $s0 = A[i]

27. A[i] = $s0

28. $s0 = A[i] + 1

29. A[i] = A[i] + $s0

30. $s0 = A[i] + B[j]

31. A[i] = B[j] - A[i]

32. $s0 = A[i+1]，要求用 addi 处理 i+1

33. A[i+2] = A[i] + $s0

34. B[4*j] = i + j；注意 B[4*j] 的字节偏移量

35. A[i] = A[i] & 15

D 组：if / if-else（目标：beq、bne、slt、j 的组合）

本组约定: 约定：a=$s0, b=$s1, c=$s2, d=$s3。只需写核心控制流，标签可自取。

36. if ($s0 == $s1) $s2 = $s2 + 1;

37. if ($s0 != $s1) $s2 = $s2 - 1;

38. if ($s0 < $s1) $s2 = $s0 + $s1;

39. if ($s0 >= $s1) $s2 = $s0 - $s1;

40. if ($s0 == 0) $s1 = $s1 | 1;

41. if ($s0 < 10) $s1 = $s1 & 15;

42. if ($s0 < $s1) $s2 = $s2 + 1; else $s3 = $s3 + 1;

43. if ($s0 == $s1) $s2 = $s2 + $s3; else $s2 = $s2 - $s3;

E 组：循环（目标：初始化、循环条件、循环体、更新、跳回）

本组约定: 约定：i=$s0, n=$s1, sum=$s2, A 基地址=$s4。word 数组。

44. for (i=0; i<n; i++) sum = sum + i;

45. while (i < n) { sum = sum + 1; i++; }

46. for (i=0; i<10; i++) sum = sum + i;

47. for (i=0; i<n; i++) sum = sum + A[i];

48. for (i=0; i<n; i++) A[i] = A[i] + 1;

49. while (A[i] != 0) { sum = sum + A[i]; i++; }

F 组：嵌套循环与真题风格应用题（目标：完整 C 转 MIPS + 注）

真题风格: 2022 真题出现过双重 for 循环：for(i=0;i<a;i++){for(j=0;j<b;j++){D[4*j]=i+j;}}。本组按同一风格训练。

50. 已知 a=$s0, b=$s1, i=$t0, j=$t1, D 基地址=$s2。翻译：for(i=0;i<a;i++){ for(j=0;j<b;j++) { D[4*j]=i+j; } }。要求关键指令注释。

51. 已知 n=$s1, i=$s0, sum=$s2, A 基地址=$s4。翻译：for(i=0;i<n;i++){ if(A[i] != 0) sum = sum + A[i]; }

52. 已知 n=$s1, i=$s0, A 基地址=$s4。翻译：for(i=0;i<n;i++){ A[i] = A[i] | 1; }

53. 已知 a=$s0, b=$s1, i=$t0, j=$t1, A 基地址=$s2。翻译：for(i=0;i<a;i++){ for(j=0;j<b;j++){ A[i] = A[i] + j; } }

G 组：函数调用与栈（目标：jal、jr、$a0/$v0/$ra/$sp ）

本组约定: 只掌握考试可能需要的基本规则：$a0-$a3 传参，$v0 放返回值，jal 会改写 $ra，函数返回用 jr $ra；嵌套调用或递归时要用 $sp 保存 $ra/参数/需要恢复的寄存器。

54. 翻译函数 int add2(int x, int y){ return x + y; }。约定 x 在 $a0, y 在 $a1, 返回值放 $v0。

55. 写出调用 add2(3,4) 的代码，并把返回值保存到 $s0。不能用 li/move。

56. 翻译函数 int inc(int x){ return x + 1; }。

57. 翻译函数 int f(int n){ return inc(n) + n; }。要求保存 $ra 和原参数 n。

58. 翻译递归框架：int fact_like(int n){ if(n<1) return 1; else return fact_like(n-1)+n; }，不要求乘法。

59. 说明为什么 f 调用 inc 前要保存 $ra。用 2-3 句话回答。

H 组：反向翻译与错误修正（目标：看懂 MIPS，发现典型坑）

60. 把代码翻译成 C：lw $t0, 8($s4); add $s0,$s1,$t0

61. 把代码翻译成 C：sll $t0,$s1,2; add $t0,$s4,$t0; lw $s0,0($t0)

62. 把代码翻译成 C：slt $t0,$s0,$s1; beq $t0,$zero,L1; addi $s2,$s2,1; L1:

63. 改错：sw $s0, 3($s4) 用于 word 数组 A[3]。

64. 改错：sll $t0,$s1,4 用于计算 word 数组 A[i] 的偏移。

65. 判断是否为基本块：add; sub; and。说明理由。

66. 判断是否为基本块：add; beq; sub。说明理由。

67. 判断是否为基本块：add; jal f; sub。说明理由。

I 组：综合小卷（按应用题标准作答）

68. 综合 1。已知 x=$s0, y=$s1, z=$s2, A 基地址=$s3，word 数组。翻译：

```
if (x < y) A[3] = x + z;
else A[4] = y - z;
```

69. 综合 2。已知 i=$s0, n=$s1, sum=$s2, A 基地址=$s3。翻译：

```
for(i=0; i<n; i++){
   sum = sum + A[i];
   A[i] = sum;
}
```

70. 综合 3。已知 a=$s0, b=$s1, i=$t0, j=$t1, C 基地址=$s2。翻译：

```
for(i=0; i<a; i++){
  for(j=0; j<b; j++){
    C[i] = C[i] + j;
  }
}
```

71. 综合 4。已知函数 g(x) 已存在，参数在 $a0，返回值在 $v0。翻译函数：

```
int f(int n){
  return g(n-1) + n;
}
```

三、答案与解析

判分说明: 答案不唯一。只要符合 MIPS 规则、寄存器使用自洽、偏移量正确、控制流等价即可得分。应用题建议给关键指令写注释。

1. $s0 = $s1 + $s2

```
add $s0, $s1, $s2
```

2. $s0 = $s1 - $s2

```
sub $s0, $s1, $s2
```

3. $s0 = $s1 + 15

```
addi $s0, $s1, 15
```

4. $s0 = $s1 - 7

```
addi $s0, $s1, -7
```

5. $s0 = ($s1 < 100)

```
slti $s0, $s1, 100
```

6. $s0 = ($s1 < -3)

```
slti $s0, $s1, -3
```

7. $s0 = $s1 & 0x00ff

```
andi $s0, $s1, 0x00ff
```

8. $s0 = $s1 | 0x000f

```
ori $s0, $s1, 0x000f
```

9. $s0 = $s1 << 2

```
sll $s0, $s1, 2
```

10. $s0 = $s1 >> 3

srl $s0, $s1, 3

11. $s0 = $s1 & $s2

```
and $s0, $s1, $s2
```

12. $s0 = $s1 | $s2

```
or $s0, $s1, $s2
```

13. $s0 = ~($s1 | $s2)

```
nor $s0, $s1, $s2
```

14. $s0 = ~$s1

```
nor $s0, $s1, $zero
```

15. $s0 = ($s1 + $s2) - ($s3 + 5)

```
add  $t0, $s1, $s2      # $t0 = $s1 + $s2
addi $t1, $s3, 5        # $t1 = $s3 + 5
sub  $s0, $t0, $t1      # $s0 = $t0 - $t1
```

16. A[0] -> $s0

```
lw $s0, 0($s4)
```

17. A[3] -> $s0

```
lw $s0, 12($s4)     # 3*4=12
```

18. $s0 -> A[5]

```
sw $s0, 20($s4)     # 5*4=20
```

19. $s0 = A[8] + $s1

```
lw  $t0, 32($s4)      # $t0 = A[8]
add $s0, $t0, $s1
```

20. A[12] = $s0 - A[4]

```
lw  $t0, 16($s4)      # $t0 = A[4]
sub $t0, $s0, $t0
sw  $t0, 48($s4)      # A[12] = result
```

21. $s0 = A[1] + A[2]

```
lw  $t0, 4($s4)
lw  $t1, 8($s4)
add $s0, $t0, $t1
```

22. A[6] = A[6] + 1

```
lw   $t0, 24($s4)
addi $t0, $t0, 1
sw   $t0, 24($s4)
```

23. B[4] = A[4]

```
lw $t0, 16($s4)
sw $t0, 16($s5)
```

24. A[10] = B[1] + B[2]

```
lw  $t0, 4($s5)
lw  $t1, 8($s5)
add $t0, $t0, $t1
sw  $t0, 40($s4)
```

25. 改错：add 不能直接操作内存地址

```
# 错误：add $s0,$s1,32($s4)
lw  $t0, 32($s4)
add $s0, $s1, $t0
```

26. $s0 = A[i]

```
sll $t0, $s1, 2       # byte offset = i*4
add $t0, $s4, $t0     # address of A[i]
lw  $s0, 0($t0)
```

27. A[i] = $s0

```
sll $t0, $s1, 2
add $t0, $s4, $t0
sw  $s0, 0($t0)
```

28. $s0 = A[i] + 1

```
sll  $t0, $s1, 2
add  $t0, $s4, $t0
lw   $s0, 0($t0)
addi $s0, $s0, 1
```

29. A[i] = A[i] + $s0

```
sll $t0, $s1, 2
add $t0, $s4, $t0
lw  $t1, 0($t0)
add $t1, $t1, $s0
sw  $t1, 0($t0)
```

30. $s0 = A[i] + B[j]

```
sll $t0, $s1, 2
add $t0, $s4, $t0
lw  $t1, 0($t0)       # A[i]
sll $t2, $s2, 2
```

```
add $t2, $s5, $t2
lw  $t3, 0($t2)       # B[j]
add $s0, $t1, $t3
```

31. A[i] = B[j] - A[i]

```
sll $t0, $s1, 2
add $t0, $s4, $t0     # &A[i]
lw  $t1, 0($t0)       # A[i]
sll $t2, $s2, 2
add $t2, $s5, $t2
lw  $t3, 0($t2)       # B[j]
sub $t1, $t3, $t1
sw  $t1, 0($t0)
```

32. $s0 = A[i+1]

```
addi $t0, $s1, 1      # i+1
sll  $t0, $t0, 2
add  $t0, $s4, $t0
lw   $s0, 0($t0)
```

33. A[i+2] = A[i] + $s0

```
sll  $t0, $s1, 2
add  $t0, $s4, $t0
lw   $t1, 0($t0)      # A[i]
add  $t1, $t1, $s0
addi $t2, $s1, 2      # i+2
sll  $t2, $t2, 2
add  $t2, $s4, $t2
sw   $t1, 0($t2)
```

34. B[4*j] = i + j

```
add $t0, $s1, $s2     # value = i+j
sll $t1, $s2, 4       # byte offset = (4*j)*4 = 16*j
add $t1, $s5, $t1
sw  $t0, 0($t1)
```

35. A[i] = A[i] & 15

```
sll  $t0, $s1, 2
add  $t0, $s4, $t0
lw   $t1, 0($t0)
andi $t1, $t1, 15
sw   $t1, 0($t0)
```

36. if ($s0 == $s1) $s2++

```
bne  $s0, $s1, End
addi $s2, $s2, 1
End:
```

37. if ($s0 != $s1) $s2--

```
beq  $s0, $s1, End
addi $s2, $s2, -1
End:
```

38. if ($s0 < $s1) $s2 = $s0 + $s1

```
slt $t0, $s0, $s1
beq $t0, $zero, End
add $s2, $s0, $s1
End:
```

39. if ($s0 >= $s1) $s2 = $s0 - $s1

```
slt $t0, $s0, $s1      # true means $s0 < $s1
```

```
bne $t0, $zero, End    # if <, skip
sub $s2, $s0, $s1
End:
```

40. if ($s0 == 0) $s1 = $s1 | 1

```
bne $s0, $zero, End
ori $s1, $s1, 1
End:
```

41. if ($s0 < 10) $s1 = $s1 & 15

```
slti $t0, $s0, 10
beq  $t0, $zero, End
andi $s1, $s1, 15
End:
```

42. if-else with <

```
slt $t0, $s0, $s1
beq $t0, $zero, Else
addi $s2, $s2, 1
j End
Else:
addi $s3, $s3, 1
End:
```

43. if-else with ==

```
bne $s0, $s1, Else
add $s2, $s2, $s3
j End
Else:
sub $s2, $s2, $s3
End:
```

44. for (i=0; i<n; i++) sum+=i

```
addi $s0, $zero, 0     # i = 0
Loop:
slt  $t0, $s0, $s1     # i < n ?
beq  $t0, $zero, End
add  $s2, $s2, $s0     # sum += i
addi $s0, $s0, 1       # i++
j    Loop
End:
```

45. while (i < n)

```
Loop:
slt  $t0, $s0, $s1
beq  $t0, $zero, End
addi $s2, $s2, 1
addi $s0, $s0, 1
j    Loop
End:
```

46. for i<10

```
addi $s0, $zero, 0
Loop:
slti $t0, $s0, 10      # i < 10 ?
beq  $t0, $zero, End
add  $s2, $s2, $s0
addi $s0, $s0, 1
j    Loop
End:
```

47. sum += A[i]

```
addi $s0, $zero, 0
Loop:
slt  $t0, $s0, $s1
beq  $t0, $zero, End
sll  $t1, $s0, 2
add  $t1, $s4, $t1
lw   $t2, 0($t1)
add  $s2, $s2, $t2
addi $s0, $s0, 1
j    Loop
End:
```

48. A[i] = A[i] + 1

```
addi $s0, $zero, 0
Loop:
slt  $t0, $s0, $s1
beq  $t0, $zero, End
sll  $t1, $s0, 2
add  $t1, $s4, $t1
lw   $t2, 0($t1)
addi $t2, $t2, 1
sw   $t2, 0($t1)
addi $s0, $s0, 1
j    Loop
End:
```

49. while A[i] != 0

```
Loop:
sll $t0, $s0, 2
add $t0, $s4, $t0
lw  $t1, 0($t0)
beq $t1, $zero, End
add $s2, $s2, $t1
addi $s0, $s0, 1
j   Loop
End:
```

50. 真题风格：D[4*j] = i+j

```
addi $t0, $zero, 0       # i = 0
Outer:
slt  $t2, $t0, $s0       # i < a ?
beq  $t2, $zero, Done
addi $t1, $zero, 0       # j = 0
Inner:
slt  $t2, $t1, $s1       # j < b ?
beq  $t2, $zero, NextI
add  $t3, $t0, $t1       # $t3 = i + j
sll  $t4, $t1, 4         # byte offset = (4*j)*4 = 16*j
add  $t4, $s2, $t4       # address of D[4*j]
sw   $t3, 0($t4)         # D[4*j] = i+j
addi $t1, $t1, 1         # j++
j    Inner
NextI:
addi $t0, $t0, 1         # i++
j    Outer
Done:
```

51. for + if: sum += A[i] if nonzero

```
addi $s0, $zero, 0
Loop:
```

```
slt  $t0, $s0, $s1
beq  $t0, $zero, End
sll  $t1, $s0, 2
add  $t1, $s4, $t1
lw   $t2, 0($t1)
beq  $t2, $zero, Skip
add  $s2, $s2, $t2
Skip:
addi $s0, $s0, 1
j    Loop
End:
```

52. A[i] = A[i] | 1

```
addi $s0, $zero, 0
Loop:
slt  $t0, $s0, $s1
beq  $t0, $zero, End
sll  $t1, $s0, 2
add  $t1, $s4, $t1
lw   $t2, 0($t1)
ori  $t2, $t2, 1
sw   $t2, 0($t1)
addi $s0, $s0, 1
j    Loop
End:
```

53. 双重循环：A[i] += j

```
addi $t0, $zero, 0       # i = 0
Outer:
slt  $t2, $t0, $s0
beq  $t2, $zero, Done
addi $t1, $zero, 0       # j = 0
Inner:
slt  $t2, $t1, $s1
beq  $t2, $zero, NextI
sll  $t3, $t0, 2         # i*4
add  $t3, $s2, $t3       # &A[i]
lw   $t4, 0($t3)
add  $t4, $t4, $t1
sw   $t4, 0($t3)
addi $t1, $t1, 1
j    Inner
NextI:
addi $t0, $t0, 1
j    Outer
Done:
```

54. int add2(int x,int y)

```
add2:
add $v0, $a0, $a1       # return x+y
jr  $ra
```

55. 调用 add2(3,4)

```
addi $a0, $zero, 3
addi $a1, $zero, 4
jal  add2
add  $s0, $v0, $zero    # $s0 = return value
```

56. int inc(int x)

```
inc:
addi $v0, $a0, 1
```

```
jr   $ra
```

57. int f(int n){ return inc(n)+n; }

```
f:
addi $sp, $sp, -8
sw   $ra, 4($sp)        # save return address of f
sw   $a0, 0($sp)        # save n
jal  inc                # inc(n), result in $v0
lw   $a0, 0($sp)        # restore n
lw   $ra, 4($sp)        # restore f's return address
addi $sp, $sp, 8
add  $v0, $v0, $a0      # return inc(n)+n
jr   $ra
```

58. 递归框架 fact_like

```
fact_like:
slti $t0, $a0, 1        # n < 1 ?
beq  $t0, $zero, Recur
addi $v0, $zero, 1      # return 1
jr   $ra
Recur:
addi $sp, $sp, -8
sw   $ra, 4($sp)
sw   $a0, 0($sp)
addi $a0, $a0, -1
jal  fact_like
lw   $a0, 0($sp)
lw   $ra, 4($sp)
addi $sp, $sp, 8
add  $v0, $v0, $a0      # return fact_like(n-1)+n
jr   $ra
```

59. 为什么保存 $ra

```
jal 会把调用者的下一条指令地址写入 $ra。如果 f 调用 inc，$ra 会被改成“从 inc 返回到 f 的地址”，从而覆盖 f 原本要返回给 main 的地址。因此嵌套调用前必须把旧 $ra 存到栈上，返回前再恢复。
```

60. 反向：lw + add

$s0 = $s1 + A[2]，其中 A 的基地址在 $s4；8 = 2*4。

61. 反向：变量下标

$s0 = A[$s1]，其中 A 的基地址在 $s4。

62. 反向：if

if ($s0 < $s1) { $s2 = $s2 + 1; }

63. 改错 word A[3]

word 数组 A[3] 偏移量应为 3*4=12，所以应写 sw $s0, 12($s4)。

64. 改错 i*4

word 数组 A[i] 的字节偏移量是 i*4，因此应写 sll $t0, $s1, 2，而不是左移 4 位。左移 4 位是 i*16。

65. 基本块：add/sub/and

是。它是一段连续顺序执行的指令，中间没有 beq/bne/j/jal/jr 等改变控制流的指令。

66. 基本块：add/beq/sub

不是一个完整基本块。beq 会改变控制流，通常基本块在 beq 处结束。

67. 基本块：add/jal/sub

通常不作为一个普通基本块整体。jal 会跳到函数执行并改写 $ra，控制流发生转移。

68. 综合 1：if-else + 数组固定下标

```
slt $t0, $s0, $s1
```

```
beq $t0, $zero, Else
add $t1, $s0, $s2
sw  $t1, 12($s3)       # A[3]
j   End
Else:
sub $t1, $s1, $s2
sw  $t1, 16($s3)       # A[4]
End:
```

69. 综合 2：for + sum + store back

```
addi $s0, $zero, 0
Loop:
slt  $t0, $s0, $s1
beq  $t0, $zero, End
sll  $t1, $s0, 2
add  $t1, $s3, $t1
lw   $t2, 0($t1)
add  $s2, $s2, $t2
sw   $s2, 0($t1)
addi $s0, $s0, 1
j    Loop
End:
```

70. 综合 3：双重循环 C[i]+=j

```
addi $t0, $zero, 0       # i=0
Outer:
slt  $t2, $t0, $s0
beq  $t2, $zero, Done
addi $t1, $zero, 0       # j=0
Inner:
slt  $t2, $t1, $s1
beq  $t2, $zero, NextI
sll  $t3, $t0, 2
add  $t3, $s2, $t3       # &C[i]
lw   $t4, 0($t3)
add  $t4, $t4, $t1
sw   $t4, 0($t3)
addi $t1, $t1, 1
j    Inner
NextI:
addi $t0, $t0, 1
j    Outer
Done:
```

71. 综合 4：函数调用 g(n-1)+n

```
f:
addi $sp, $sp, -8
sw   $ra, 4($sp)
sw   $a0, 0($sp)
addi $a0, $a0, -1
jal  g
lw   $a0, 0($sp)
lw   $ra, 4($sp)
addi $sp, $sp, 8
add  $v0, $v0, $a0
jr   $ra
```

附录：C 转 MIPS 评分自查清单

检查点	自查问题
寄存器约定	题目给的 a,b,i,j,A 基地址是否按要求使用？临时值是否放在 $t?
内存访问	add/sub/and/or 有没有错误地直接使用 32($s3)？如果有，必须先 lw。
偏移量	word 数组是否下标乘 4？D[4*j] 是否计算为 16*j 字节?
变量下标	是否先 sll i,2，再 add 基地址?
条件判断	< 是否用 slt+beq/bne？==/!= 是否直接用 beq/bne?
循环结构	是否有初始化、判断、循环体、更新、j 回循环头、Exit 标签?
if-else	then 部分后是否 j End 避免落入 else?
函数调用	嵌套调用/递归前是否保存 $ra 和必要参数？返回值是否放 $v0?
基本块	遇到 beq/bne/j/jal/jr 是否断开基本块?

完成本练习后，你应能独立完成真题风格的 C 代码转 MIPS 应用题。